Developers Implementation Guide

Git Workflow & Pull Request Standard

V1.0

Document Summary

Provider	Montu Tech
Effective Date	Aug 30, 2026
Classification	Internal
Version	V1.0

Version Control

Version	Description	Owner
1.0	Initial Release	Omar Alaa

Document Reviewers

Owner	Title
Omar Alaa	Product Owner

Table of Contents

Document Summary 2
Version Control 2
Document Reviewers 2
Table of Contents **3**
1. Introduction & Overview 4
2. Branching Strategy 4
Examples: 4
4. Commit Message Guidelines 5
5. Pull Request (PR) Standards & Lifecycle 5
6. Workflow Summary for Interns 5

1. Introduction & Overview

Welcome to Montu's team. This document outlines our standard Git branching strategy, commit message conventions, and Pull Request (PR) lifecycle. Adhering to these guidelines ensures clean code management, traceability with ClickUp tasks, and a smooth deployment pipeline across our development, staging, and production environments.

2. Branching Strategy

We follow a structured branching hierarchy to manage feature development, testing, and production releases. Our main persistent branches and their specific responsibilities are defined below:

Branch Name	Responsibility & Purpose	Who Merges Into It
dev	Primary integration branch for developers. All new features and bug fixes land here first.	Developers via feature branches
UAT	Staging and testing branch. Once the dev branch is stable, code is promoted here for QA and client/internal testing.	Team Leads / Senior Engineers from dev
prod	Production-ready branch containing code validated in UAT, awaiting final deployment to live environments.	Tech Leads / DevOps from UAT
main / master	The ultimate source of truth representing the current live production release state.	Tech Leads from prod

3. Branch Naming Conventions

To maintain clear context and direct linkage to our task management boards, all working branches created off dev must follow a strict naming convention incorporating the ClickUp task ID, the type of work, and a brief description.

- Format: [type]/CU-[Task_ID]-[short-description]
- Allowed Types:
    - feature: For new developments, components, or functionality.
    - bugfix: For fixing defects, errors, or unexpected behaviors.
    - hotfix: For critical production patches.
    - chore: For maintenance, refactoring, or dependency updates.

Examples:

feature/CU-1.2-auth-login-screen
bugfix/CU-3.2-payment-gateway-timeout
chore/CU-2.4-update-tailwind-config

4. Commit Message Guidelines

Commit messages must be concise, descriptive, and explicitly linked to our ClickUp tracking system. This allows automated integrations and team members to trace code changes back to specific requirements.

- Format: [type](CU-[Task_ID]): short summary of changes
- Examples:
    - feat(CU-1.2): add form validation to login input fields
    - fix(CU-3.2): resolve null pointer exception on checkout callback
    - refactor(CU-4.5): clean up unused CSS classes in navbar

5. Pull Request (PR) Standards & Lifecycle

Pull requests are the gateway for code review and quality assurance before code propagates from dev to UAT and ultimately prod.

1. Target Branch: Feature branches must always target the dev branch. Direct PRs to UAT or prod are restricted to senior engineers.
2. PR Title: Must mirror the branch naming convention, including the ClickUp ID (e.g., [Feature] CU-1.2 - Implement Login Screen UI).
3. PR Description Template: Every PR must include:
    - ClickUp Link: Direct hyperlink to the corresponding task.
    - Overview of Changes: Bullet points summarizing what was implemented or fixed.
    - Testing Instructions: Steps for reviewers and QA to verify the changes locally or in preview environments.
    - Screenshots / Recordings: Visual proof for frontend changes or UI updates.
4. Review Requirements:
    - At least one peer review approval is required for interns.
    - All automated CI/CD checks, linters, and build tests must pass successfully.
    - No merge conflicts with the target branch are permitted.

6. Workflow Summary for Interns

- Pull the latest updates from the dev branch.
- Create a properly named feature branch incorporating your ClickUp task ID.
- Commit frequently using descriptive, task-linked commit messages.
- Push your branch and open a Pull Request targeting dev with a complete description.
- Address review feedback, merge once approved, and update your ClickUp task status accordingly.